Lecture 8: Data Storage and File Structures

Why Storage Internals Matter

Where storage decisions surface in practice

- **OLTP systems:** latency depends on whether requests hit hot pages in RAM or trigger random I/O.
- **Analytics systems:** file format and layout determine whether a query scans full rows or only the needed columns.
- **Modern AI data pipelines:** embeddings, logs, and feature tables often span DRAM, SSD, and object storage tiers.

The Core Truth

You cannot optimize SQL queries, design scalable schemas, or reason about reliability without understanding how tuples, pages, and logs are physically laid out on storage.

From Logical Design to Physical Reality

We've mastered the art of designing a clean, normalized schema. Now, we ask: how does a DBMS actually *store* the data on physical hardware?

Our Journey So Far

Logical World: Schemas, Relations, Tuples, SQL, Normal Forms. This is an abstraction that helps us reason about data cleanly.

Today's Focus: The Physical World

Physical World: Files, Blocks, Bytes, Disk Heads, Rotational Delays, and SSD Caching. This is the messy reality of hardware constraints.

The Big Question

How do we efficiently map the clean logical model onto the complex and slow physical layer? This is the central challenge of database storage management.

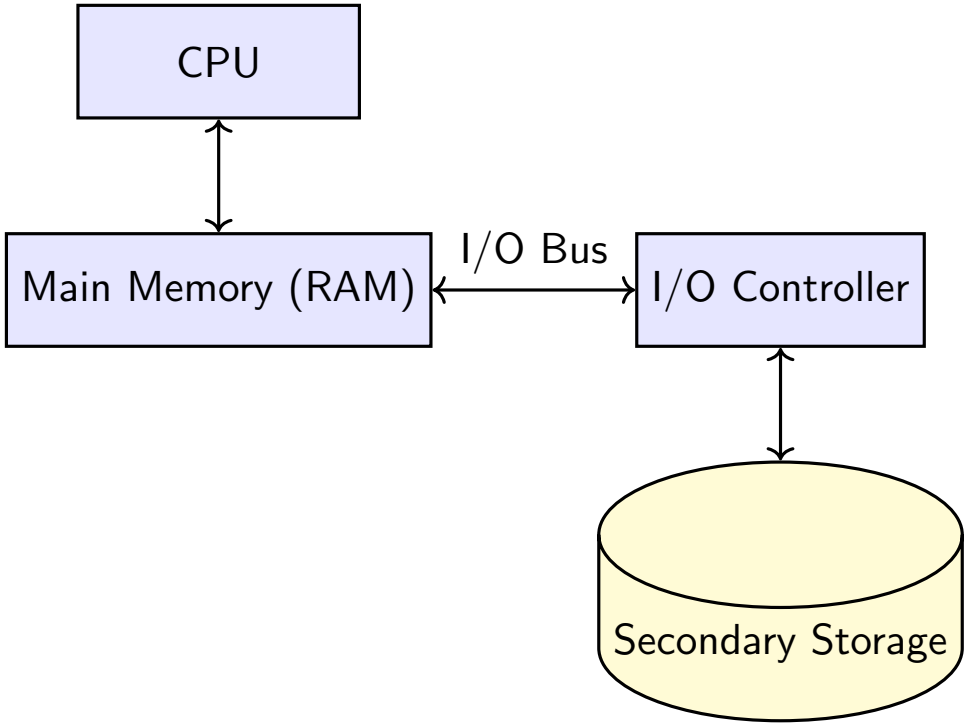
Agenda: The Pedagogical Path

1. **Part 1: Foundational Storage Mechanics**
   - The Storage Hierarchy and the Von Neumann Bottleneck
   - Disk Mechanics (HDD vs. NVMe SSD tiers)
   - Record Layout and Block Organization
2. **Part 2: Storage Layout Inside the DBMS**
   - The Buffer Manager, Replacement Policies, and WAL
   - Slotted Pages, Logical Indirection, TOAST, and MVCC
   - Row Stores, Column Stores, and PAX Hybrid Layout
   - Preview: how physical storage choices motivate later index lectures
3. **Modern Directions:** ZNS SSDs, CXL Memory, Columnar Formats (Parquet / Iceberg)

Part 1: Foundational Storage Mechanics

The Context: Computer Architecture

Databases run on standard hardware, governed by the Von Neumann architecture. Data must traverse the bus to be processed.

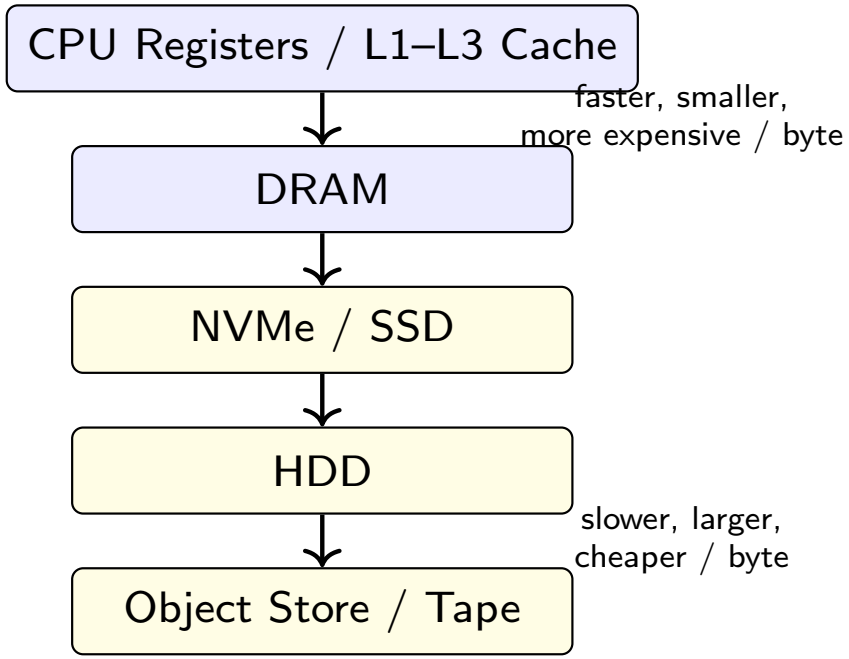


- The **CPU** operates only on data currently inside its internal registers or L1/L2 caches.
- Data must be actively moved from **Secondary Storage** into **Main Memory**.
- This mechanical and electrical movement across the **I/O Bus** represents the single biggest bottleneck in data-intensive systems.

The Memory Hierarchy

Storage Hierarchies: cache frequently accessed data on faster storage to improve performance.

- **Primary storage** (volatile)
  - CPU Registers & Cache (L1/L2/L3)
  - Main Memory (DRAM)
- **Secondary storage** (persistent)
  - Flash / NVMe SSD
  - Magnetic Disk (HDD)
- **Tertiary storage**: object stores, tapes



The Memory Hierarchy & The Immense Latency Gap

We cache frequently accessed data on faster storage tiers to mitigate astronomical latency differences.

Latency — order of magnitude (2024)

Level	Typical	Analogy
L1 Cache	∼0.5 ns	desk
DRAM	∼100 ns	bookshelf
PCIe Gen 4 NVMe	∼50-100 µs	moon
SATA SSD	∼100 µs	solar system
HDD Seek	∼10 ms	Pluto

QD1 random read, consumer drives. Enterprise NVMe: ∼20 µs.

Jim Gray's Analogy

- Cache → desk
- Memory → bookshelf
- **NVMe → moon**
- **HDD → Pluto**

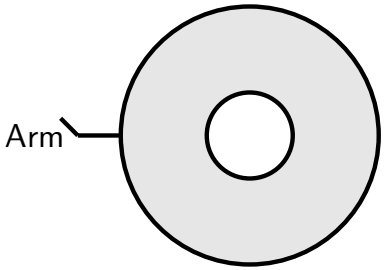
The I/O Cost Model

Because disk access dwarfs CPU time, we simplify analysis: **count the number of disk I/Os** and ignore CPU processing time. With Gen 5 NVMe the CPU can become the bottleneck again, but the I/O model remains the correct starting point.

The Contenders: HDD vs. SSD Tiers

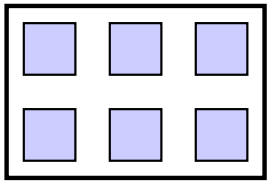
Secondary storage spans a wide performance spectrum. Choosing the right tier is a cost–performance trade-off.

Hard Disk Drive (HDD)



- **Mechanical**: rotating platters + actuator arm.
- **Slow random access**: dominated by seek time (∼8 ms) and rotational delay.
- Use case: cold storage, archival, backups.

SSD Tiers (2025)



Interface	Seq. speed	Rand. lat.
SATA SSD	550 MB/s	∼100 µs
PCIe Gen 4 NVMe	7 GB/s	∼50 µs
PCIe Gen 5 NVMe	**14 GB/s**	**∼50 µs**

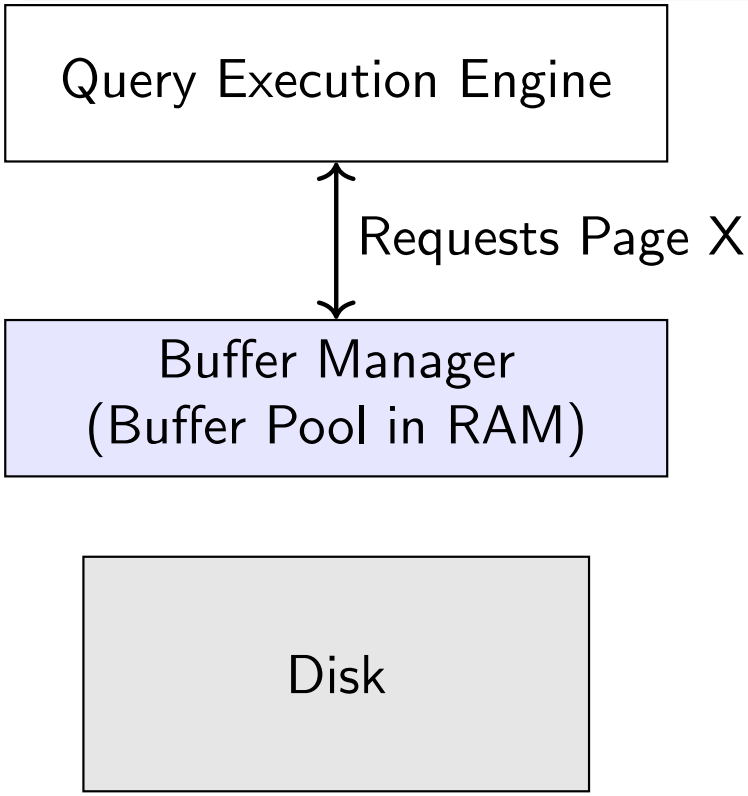
- **Electronic**: NAND flash. Zero moving parts.
- Required for transactional OLTP and AI/ML workloads.

The Bridge: The Buffer Manager

How does the DBMS manage the flow of data between slow disk and fast RAM?

The Buffer Manager

A component within the DBMS that manages a region of main memory called the **buffer pool**. All disk read/write requests go through it.

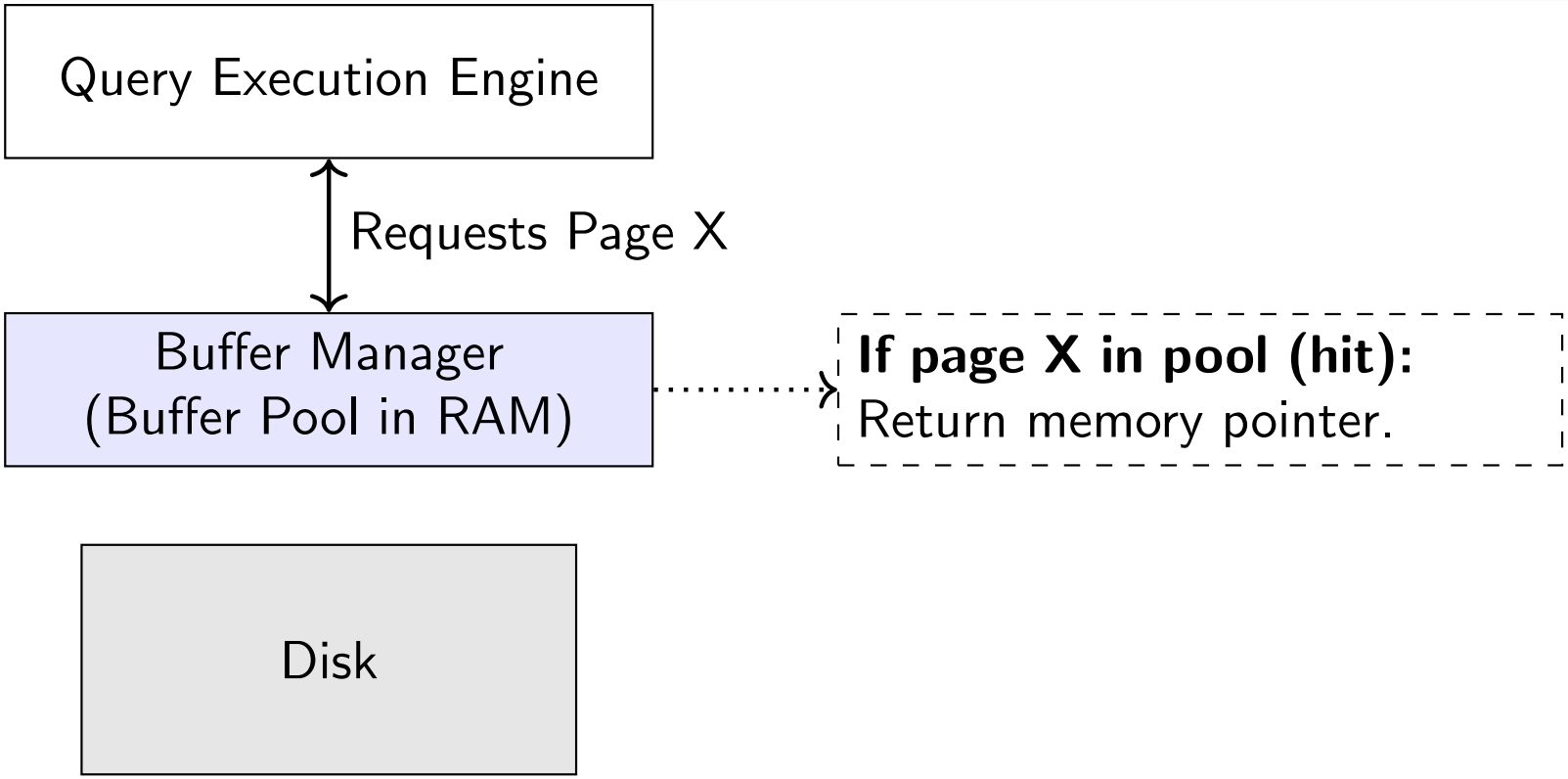


The Bridge: The Buffer Manager

How does the DBMS manage the flow of data between slow disk and fast RAM?

The Buffer Manager

A component within the DBMS that manages a region of main memory called the **buffer pool**. All disk read/write requests go through it.

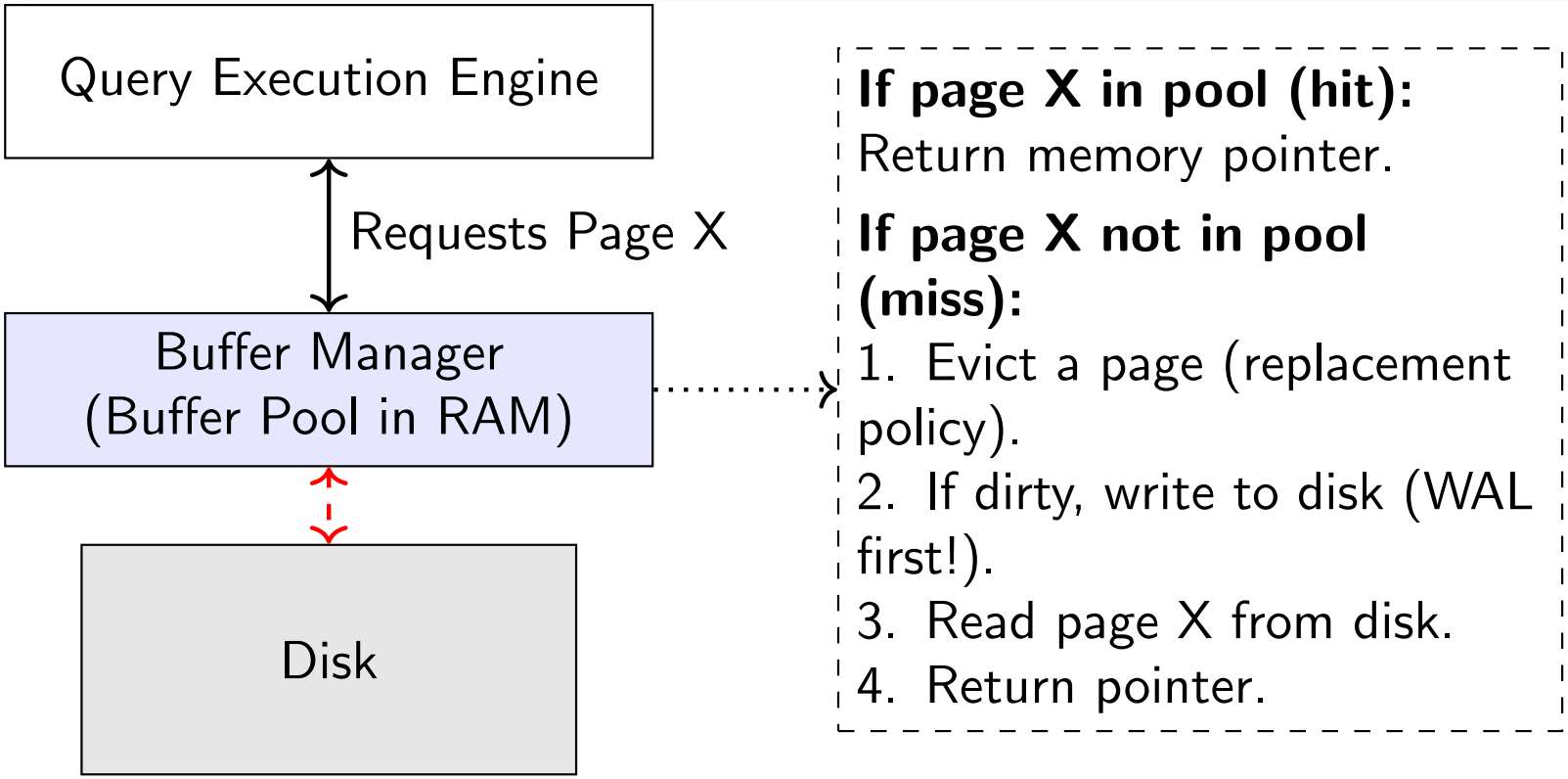


The Bridge: The Buffer Manager

How does the DBMS manage the flow of data between slow disk and fast RAM?

The Buffer Manager

A component within the DBMS that manages a region of main memory called the **buffer pool**. All disk read/write requests go through it.



Buffer Replacement Policies

LRU

- Evict the page unused for the longest time.
- **Problem:** sequential full-table scans flood the pool with single-use pages (*cache pollution*).

LRU-K

- Track the last K accesses per page.
- Evict the page with the oldest K-th access.
- Far more resistant to scan pollution than LRU.

Clock / Second-Chance

- $O(1)$ LRU approximation: one *reference bit* per frame.
- Sweep: bit=1 $\rightarrow$ reset; bit=0 $\rightarrow$ evict.
- Classic textbook algorithm. Linux now uses **MGLRU** (multi-gen LRU).

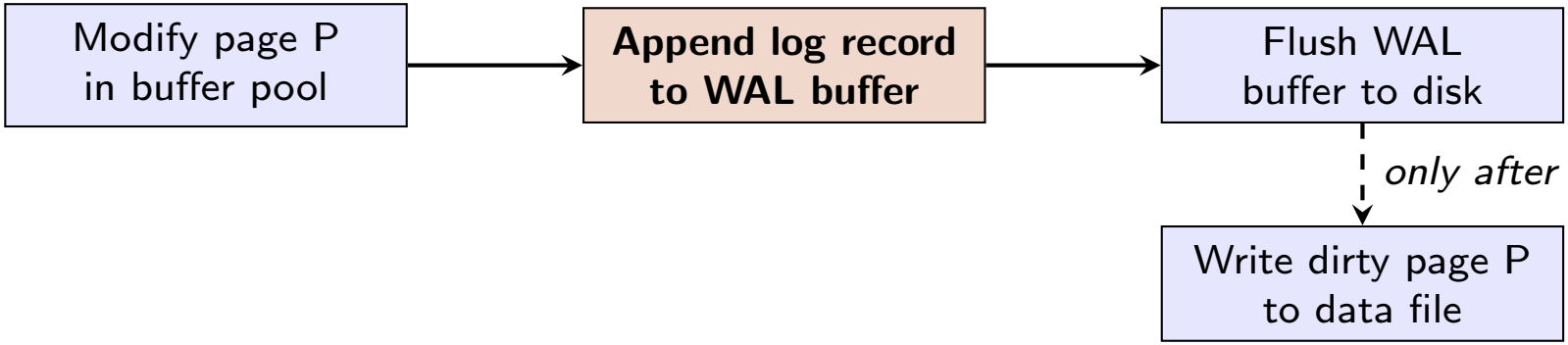
PostgreSQL in practice

- **Clock-sweep** with `usage_count` 0–5 for the main shared-buffer pool.
- **Ring buffer** isolates bulk ops (sequential scans, VACUUM, COPY) so they cannot evict hot OLTP pages.

Write-Ahead Logging (WAL)

The Rule — enables **Steal / No-Force**

- **Before** a dirty page reaches the data file, its log record must be on stable storage.
- **Steal:** dirty pages *may* be evicted before commit → flexible buffer management.
- **No-Force:** dirty pages need *not* flush at commit → high write throughput.



Write-Ahead Logging — Recovery & Systems

ARIES recovery (3 phases)

1. **Analysis** — scan WAL from last checkpoint; identify dirty pages and active transactions at crash.
2. **Redo** — replay all logged operations; restore pre-crash state.
3. **Undo** — roll back uncommitted transactions.

Real-world WAL locations

- **PostgreSQL:** `pg_wal/` directory
- **InnoDB 8.0+:** `#ib_redo0`, `#ib_redo1`...
- **SQLite WAL mode:** `db-wal` file
- **LSM-Trees:** WAL written before Memtable update

Key guarantee

No committed transaction is ever lost, even if the server crashes immediately after commit.

Classic Principle: The Five-Minute Rule

The Rule (Gray & Putzolu, SIGMOD 1987)

If a block is accessed more often than the **break-even interval**, it is cheaper to keep it in RAM than to re-fetch it from disk.

Illustrative cloud-pricing example

- **ElastiCache RAM:** ~$18 / GB / month
- **S3 object read:** ~$0.0000004 / GET

⇒ Break-even depends on **object size** and **access frequency**. We calculate it in the next exercise.

Why the gap has widened

- Object storage is now orders of magnitude cheaper per byte than RAM.
- But RAM latency is orders of magnitude lower.
- The economic model — not the numbers — is what transfers across deployments.

Gray & Putzolu (1987). The 5-minute rule for trading memory for disc accesses. SIGMOD Record, 16(3). Cloud prices: 2025 AWS published rates.

Exercise

Scenario: Caching on the Cloud (illustrative prices)

- **In-Memory Cache (RAM):** Costs **$18.00 per GB per month**.
- **Object Storage (Disk):** Costs **$0.0000004 per read request**.

For a 1 MB data object, what is the break-even number of reads per month? How does this translate to an access interval?

Exercise — Solution

Solution:

1. **Cost to cache 1 MB in RAM for 1 month:**
   $(\$18.00/1024\,\text{MB}) = \$\mathbf{0.0176}$
2. **Cost to fetch from S3 N times:**
   $N \times \$0.0000004 = \$\mathbf{0.0000004} \times \mathbf{N}$
3. **Break-even N:**
   $\$0.0176 = \$0.0000004 \times N \implies N = \mathbf{44{,}000}$ requests/month.
4. **Access interval:**
   $(30 \times 24 \times 60\,\text{min})/44{,}000 \approx$ **0.98 minutes/request**.

Conclusion

If the 1 MB object is accessed **more than once per minute**, it is cheaper to keep it in RAM. Objects accessed less frequently should stay in S3 and be fetched on demand. This economic reasoning guides buffer pool sizing and caching tier decisions in every large-scale system.

Part 1 Recap & Part 2 Preview

Part 1: Physical Reality

- **I/O Cost Model** — count disk accesses; CPU time is secondary.
- **NVMe** ∼50–100 µs; **HDD** ∼10 ms — orders of magnitude apart.
- **Buffer Manager** intercepts all I/O; WAL guarantees crash durability.
- **Replacement policies:** Clock-sweep + ring buffer (PostgreSQL).
- **Double buffering** overlaps I/O with computation.

Part 2 Preview: Storage Layout

- **Slotted pages** — variable-length records, stable RIDs.
- **TOAST, MVCC, & VACUUM** — PostgreSQL storage realities.
- **Row / Column / PAX** — locality and workload fit.
- **Parquet / Iceberg** — modern file layouts on object storage.
- **Access methods preview only** — B^+-Tree, hashing, and LSM internals come later.

2. The Physics of Storage

The Cost of an HDD I/O Example

Total Latency = Seek Time + Rotational Delay + Transfer Time

Scenario

Consider a disk with:

- Average Seek Time: **8 ms**
- Rotation Speed: **7200 RPM**
- Transfer Rate: **100 MB/s**
- Block Size: **8 KB**

The Cost of an HDD I/O Example

Total Latency = Seek Time + Rotational Delay + Transfer Time

Scenario

Consider a disk with:

- Average Seek Time: **8 ms**
- Rotation Speed: **7200 RPM**
- Transfer Rate: **100 MB/s**
- Block Size: **8 KB**

Calculation for one random block read:

1. **Seek Time**: **8.0 ms**.
2. **Rotational Delay**: Average is half a rotation.
   $(60{,}000\,\text{ms/min})/(7200\,\text{rev/min})/2 = \mathbf{4.17\ ms}$.
3. **Transfer Time**: Time to read 8 KB.
   $(0.008\,\text{MB})/(100\,\text{MB/s}) = 0.00008\,\text{s} = \mathbf{0.08\ ms}$.
4. **Total Average Latency** $= 8.0 + 4.17 + 0.08 = \mathbf{12.25\ ms}$.

I/O Optimization: Prefetching & Double Buffering

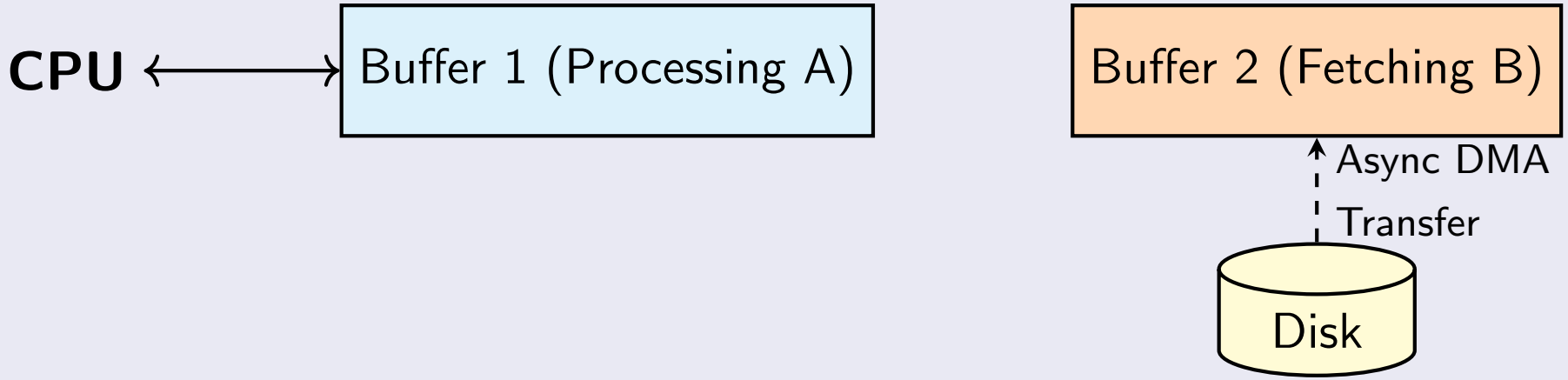
How do we prevent the CPU from idling while waiting for the mechanical disk to rotate?

The Single Buffering Bottleneck

With one buffer, execution is strictly synchronous.
Total Time = I/O + Processing + I/O + Processing...

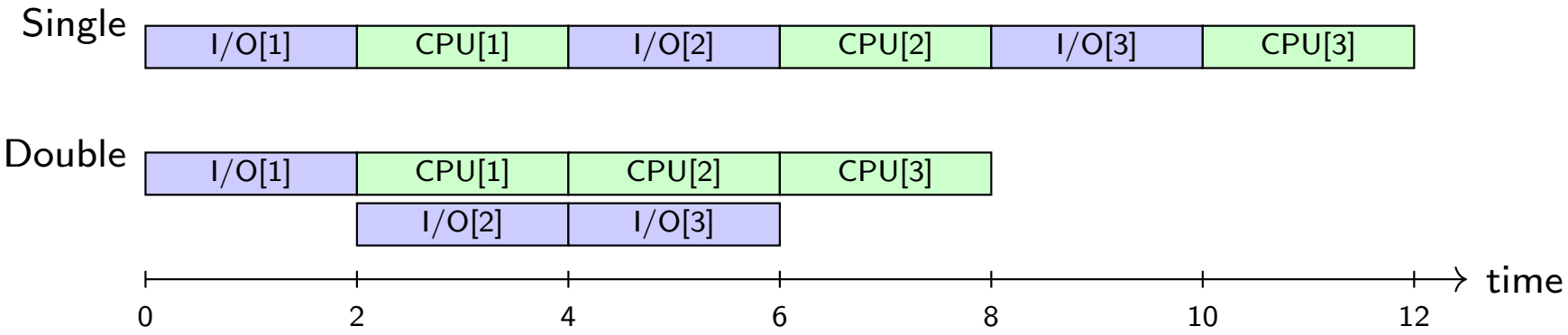
Double Buffering (Overlapping I/O)

While the CPU processes Block A from Buffer 1, the DMA controller already fetches Block B from disk into Buffer 2.



Double Buffering: Timing & Model

Gantt view (P = processing time per block, R = I/O time):



Strategy	Total Time (approx.)
Single Buffering	$N \times (P + R)$
Double Buffering	$R + N \times \max(P, R) + P$

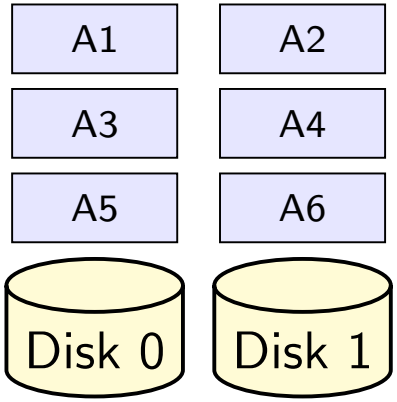
Conclusion

If $P \geq R$ (CPU-bound), I/O latency is **entirely hidden** behind computation. Effective cost per block reduces to just P.

RAID: Combining Storage Devices

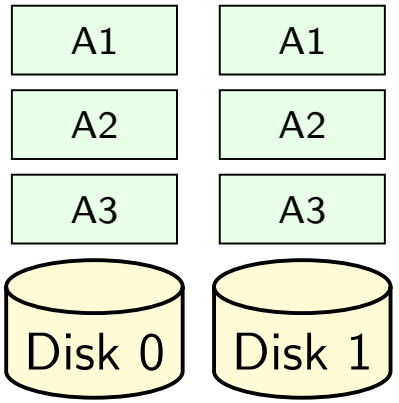
RAID (Redundant Array of Inexpensive Disks) uses multiple disks in parallel to improve throughput and/or reliability.

RAID 0 (Striping)



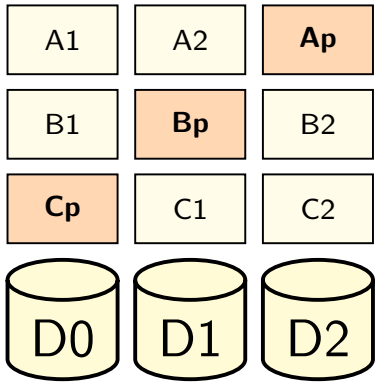
+ Max throughput. - Zero fault tolerance. One failure ⇒ all data lost.

RAID 1 (Mirroring)



+ Survives 1 disk failure. - 50 % storage efficiency.

RAID 5 (Parity)



+ Survives 1 failure, better efficiency than RAID 1. - Write penalty (XOR parity). **Warning:** unsafe on drives >4 TB — use RAID 6.

Modern context: RAID 6 and Erasure Coding

RAID 6 (dual parity) survives 2 simultaneous failures — the minimum for modern large HDDs. Cloud stores (S3, GCS) use **Reed–Solomon erasure coding**: a generalization of RAID parity configurable for k failures across n nodes.

Exercise: RAID Trade-offs

Scenario

You have **4 disks**, each with **4 TB** capacity and **200 MB/s** sequential read throughput.

1. Calculate the **usable capacity** and **max sequential read throughput** for RAID 0, RAID 1, and RAID 5.
2. Which level tolerates a single disk failure?
3. Your workload is a write-heavy OLTP database. Which level would you choose and why?

Exercise: RAID Trade-offs

Scenario

You have **4 disks**, each with **4 TB** capacity and **200 MB/s** sequential read throughput.

1. Calculate the **usable capacity** and **max sequential read throughput** for RAID 0, RAID 1, and RAID 5.
2. Which level tolerates a single disk failure?
3. Your workload is a write-heavy OLTP database. Which level would you choose and why?

Answers:

Level	Usable	Read speed	Fault tolerant?
RAID 0	16 TB	800 MB/s	No
RAID 1	8 TB	200–400 MB/s*	Yes (1 failure)
RAID 5	12 TB	600 MB/s	Yes (1 failure)

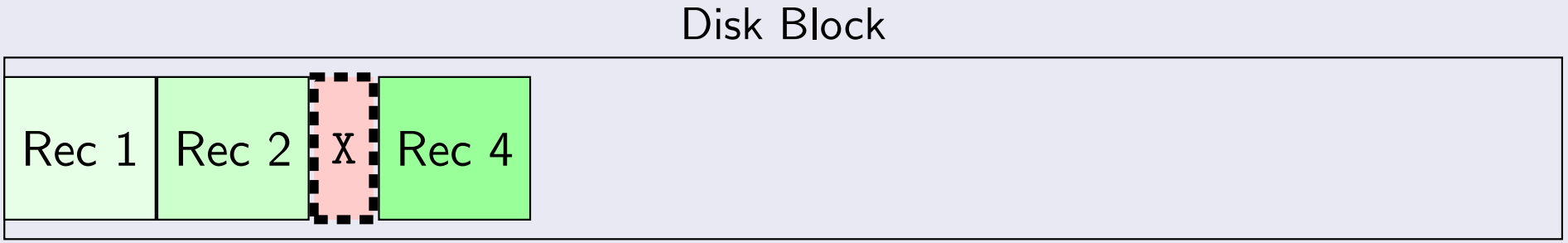
*RAID 1 sequential read = 400 MB/s only if the controller load-balances reads across both mirrors; otherwise 200 MB/s.

OLTP choice: RAID 10 (RAID 1 + RAID 0 striped mirror) is typical — combines fault tolerance with low write penalty. RAID 5 write amplification hurts OLTP latency.

3. The Data Blueprint

Packing Variable-Length Records into a Block

Naive approach: sequential packing



Deletion

Deleting Rec 3 leaves a **hole** — internal fragmentation.

Update

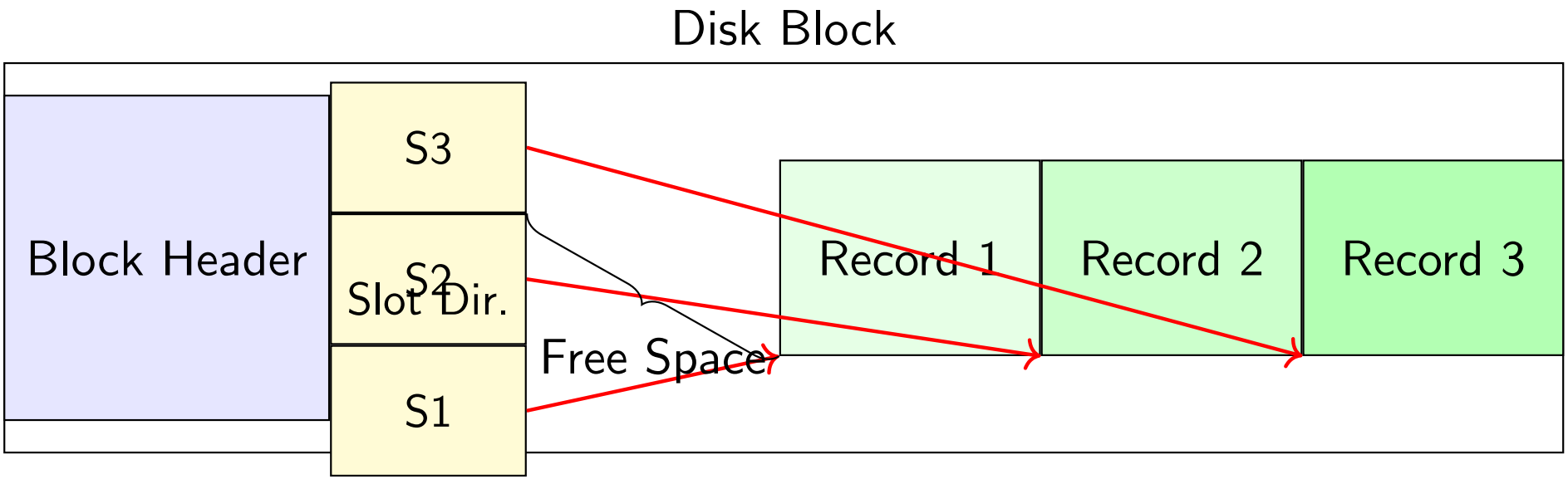
If Rec 1 grows, every subsequent record must **shift** — $O(N)$ cost.

Pointers

Physical pointers to Rec 4 become **invalid** after any shift.

$\Rightarrow$ We need a smarter layout. **Slotted pages** solve all three problems.

The Solution: The Slotted Page Structure

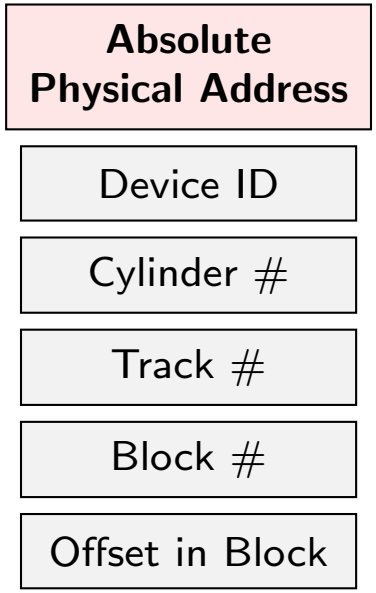


- Records grow inward from the **right**; the slot directory grows from the **left**.
- Records can be moved to compact free space. Only the small pointer in the slot entry needs updating.
- External pointers use a stable **logical Record ID (RID)** = (BlockID, Slot#). The RID never changes even when the record moves within the page.

Record Addressing: The Imperative of Indirection

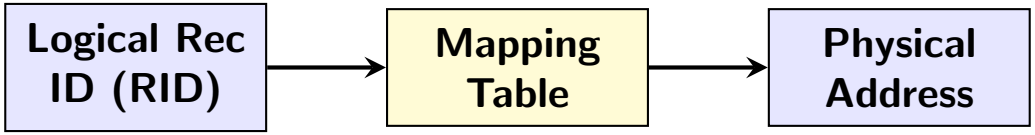
How do secondary indexes refer to a specific tuple?

Purely Physical Addressing



- **Fast** direct access.
- **Brittle**: any record move requires exhaustive update of all secondary index pointers.

Fully Indirect Addressing (Modern)



- **Flexibility:** Records can be freely shifted. Only the local slot directory updates.
- **Trade-off:** One extra pointer dereference per lookup — increases CPU cache misses slightly.
- **Used by:** PostgreSQL (heap + RID), InnoDB (primary key as logical pointer in secondary indexes).

Advanced Record Layout: Spanning & Deletion

Spanned records

- A record larger than one block is **split across pages** with a pointer chain.
- **PostgreSQL avoids spanning** heap tuples entirely.
- Oversized attributes use **TOAST**: stored out-of-line in a separate table; main heap stays compact.

Deletion & VACUUM

- Delete → tuple **marked dead** in its header; no immediate data movement.
- Dead space accumulates → **fragmentation**.

VACUUM marks dead space *reusable*; does *not* compact the file.

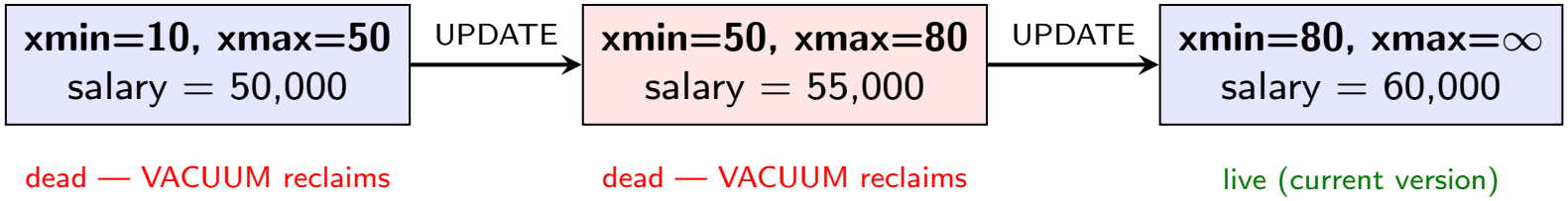
VACUUM FULL rewrites table to a new compacted file; requires exclusive lock.

autovacuum runs VACUUM automatically to prevent bloat and XID wraparound.

MVCC: Row Versioning in PostgreSQL

Core idea

Keep **multiple heap tuple versions**. Each carries `xmin`/`xmax` IDs. Readers get a consistent snapshot; writers append new versions without blocking reads.



UPDATE storage behaviour

- Normally writes a **new tuple** anywhere in the heap.
- **HOT** exception: if new version fits on the *same page* and no indexed column changed → chain within page, skip index update.

VACUUM is mandatory

- Marks dead-tuple space **reusable**.
- Advances the oldest-XID horizon.
- Prevents **transaction ID wraparound** — a critical operational failure.

4. The Grand Design: File Organization

Locality Example: Row Stores vs. Column Stores

Physical locality dictates which items are accessed together. The workload determines the optimal format.

Row Store (OLTP)

Alex	**20**	**CA**
Bob	30	CA
Carol	42	NY
David	21	MA

Stored contiguously in **one file**

- **Pro:** Full row fetch = 1 random I/O.
- **Con:** Column scan (e.g., AVG(salary)) reads all columns — massive I/O waste.

Column Store (OLAP)

Name	Age	State
Alex	20	CA
Bob	30	CA
Carol	42	NY
David	21	MA

Each column is a **distinct file**

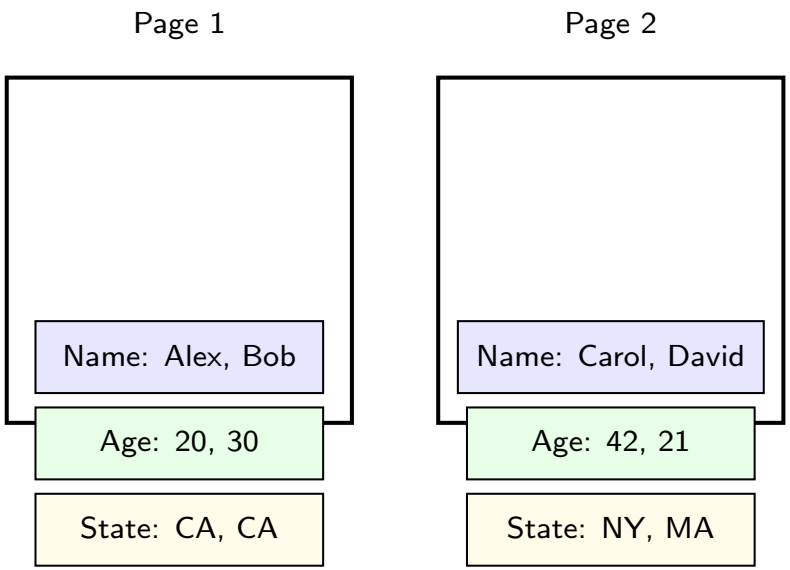
- **Pro:** Column scan reads only one file. Enables run-length encoding (RLE) and SIMD vectorization.
- **Con:** Point queries (fetch one full row) require reading N separate column files — very expensive.

The Best of Both Worlds: PAX Hybrid Layout

The idea (Ailamaki et al. 2001)

Within each disk **page**, store attributes as **mini-columns** (minipages). Pages themselves are organised row-by-row.

⇒ One page I/O fetches all columns for its rows; within the page, column data is contiguous for SIMD.



OLAP benefit

Mini-columns fit in CPU cache lines. SIMD processes them without row-striding. RLE/delta compression works well on homogeneous column data.

OLTP benefit

Full-row fetch = one page I/O (all columns co-located).

Modern influence

PAX directly inspired **Apache Arrow** (in-memory) and **Parquet row groups** (on-disk) — both apply the same minipage principle at different scales.

Comparing Physical Organization Strategies (Preview)

Strategy	Heap File	Sorted File	B^+-Tree	LSM-Tree
Write	$O(1)$ append	$O(N)$ shift	$O(\log N)$	$O(1)$ mem write
Point Read	$O(N)$ scan	$O(\log N)$	$O(\log N)$	Good + Bloom
Range Read	$O(N)$ scan	Excellent	Excellent	Okay (merge)
Workload	OLTP staging	OLAP read-only	OLTP general	Write-heavy
Systems	PostgreSQL heap	(rarely alone)	InnoDB, SQLite	Cassandra, RocksDB

Why this is only a preview

This lecture focuses on **pages, records, locality, and logging**. Access-method internals — B^+-tree splits, hash overflow handling, and LSM compaction policies — are deferred to later lectures. Here, the key point is that every organization ultimately sits on top of the same page and buffer abstractions.

What This Lecture Intentionally Leaves for Later

Storage questions answered here

- How pages are laid out and addressed
- How the buffer manager moves pages between disk and RAM
- Why WAL, MVCC, and VACUUM shape physical storage behaviour
- Why row, column, and hybrid layouts produce different I/O costs

Access-method questions deferred

- B^+-tree node splits, merges, and range-search mechanics
- Hash directory growth, overflow buckets, and dynamic hashing
- LSM compaction policies, Bloom filters, and level sizing

The bridge between lectures

Regardless of the access method, the implementation still depends on the same foundations from this lecture: **fixed-size pages, stable record references, buffer replacement, WAL, and locality-aware I/O**.

5. Modern Storage Directions

Modern Storage Directions

Zoned Namespace (ZNS) SSDs

- Internal layout exposed as **zones** requiring sequential writes.
- DBMS controls placement directly → bypasses SSD controller garbage collection.
- Natural fit for **LSM-Trees**: append-only writes align with zone constraints.
- Result: higher, more **predictable throughput**.

CXL Memory Pooling

Intel Optane discontinued 2022. **CXL** (Compute Express Link) is the emerging standard:

- PCIe 5.0-based; byte-addressable memory expansion.
- Enables **memory pooling across servers**.
- Samsung, SK Hynix CXL modules entering data centers 2025.

Caution: CXL reduces overhead vs. network RPC, but latency, NUMA topology, and software support still matter. Rack-scale pooling remains aspirational.

Columnar Formats: Parquet, Delta Lake & Iceberg

Apache Parquet — *file* format

- Used by Spark, Trino, BigQuery, DuckDB.
- **Row groups** → **column chunks** → **pages**: PAX-like hierarchy.
- Dictionary encoding + RLE per column.
- **Predicate pushdown**: skip whole row groups via per-column min/max stats.

Delta Lake vs. Iceberg — *table* formats

- **Delta Lake**: versioned Parquet files + JSON/Parquet **transaction log**. ACID via optimistic concurrency.
- **Iceberg**: **snapshot-based**; atomic metadata pointer swap. Serializable isolation; designed for concurrent writers.
- Both: time-travel, schema evolution, partition pruning on object stores.

Connection to this lecture

Parquet row groups are *PAX-like* at file scale. I/O cost analysis, buffer management, and columnar locality all apply directly to these formats.

Impact on SQL Query Plans

The physical storage layer directly dictates the choices made by the query optimizer.

Query: `SELECT title FROM Movies WHERE year = 1999;`

The query plan depends entirely on the file organization:

- **Heap File**: **Full Table Scan** — every block must be read.

Impact on SQL Query Plans

The physical storage layer directly dictates the choices made by the query optimizer.

Query: `SELECT title FROM Movies WHERE year = 1999;`

The query plan depends entirely on the file organization:

- **Heap File**: **Full Table Scan** — every block must be read.
- **Sorted by** `year`: **Range Scan** — binary search to the first 1999 entry, then sequential scan until year changes.

Impact on SQL Query Plans

The physical storage layer directly dictates the choices made by the query optimizer.

Query: `SELECT title FROM Movies WHERE year = 1999;`

The query plan depends entirely on the file organization:

- **Heap File**: **Full Table Scan** — every block must be read.
- **Sorted by** `year`: **Range Scan** — binary search to the first 1999 entry, then sequential scan until year changes.
- **B^+-Tree index on** `year`: **Index Range Scan** — a few I/Os to descend the tree, then a sequential leaf scan. We return to the tree internals in the later indexing lecture.

Impact on SQL Query Plans

The physical storage layer directly dictates the choices made by the query optimizer.

Query: `SELECT title FROM Movies WHERE year = 1999;`

The query plan depends entirely on the file organization:

- **Heap File**: **Full Table Scan** — every block must be read.
- **Sorted by** `year`: **Range Scan** — binary search to the first 1999 entry, then sequential scan until year changes.
- **B^+-Tree index on** `year`: **Index Range Scan** — a few I/Os to descend the tree, then a sequential leaf scan. We return to the tree internals in the later indexing lecture.
- **Column Store / Parquet**: **Column Scan** — reads only the `year` and `title` columns; entire row groups where min/max(`year`) excludes 1999 are skipped.

Impact on SQL Query Plans — The Punchline

Physical layout determines the query plan cost.
The best SQL query in the world is helpless against a poor physical layout.

OLTP vs OLAP layout choice

- **OLTP** (point queries) → heap storage plus an index structure.
- **OLAP** (column scans) → column store or Parquet.
- **Hybrid** → PAX within pages, partitioned Parquet on disk.

Schema design is a storage decision

- Heavy normalisation → many random I/Os to reconstruct a business object.
- Strategic denormalisation + Parquet → single sequential column scan for analytical queries.